

Graph Neural Networks

Universidade Federal do Pará (UFPA)

Contributors:

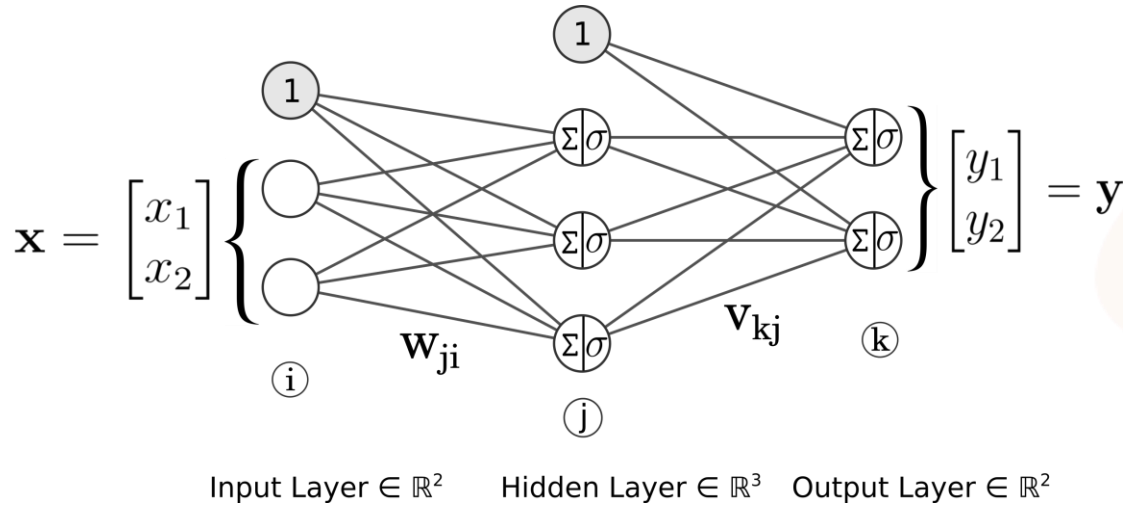
**Prof. Aldebaro Klautau
Cláudio Modesto**

www.lasse.ufpa.br & www.ppgee.ufpa.br

Mathematics of Classical Deep Learning

Forward Propagation

The process used to generate the output of a neural network



Keynote of this process:

- Weight initialization
- The mathematical computation

Weight Initialization Techniques	Activation Function Match
Xavier/Glorot ¹	Tanh, Softmax
He	ReLU and Variants
LeCun	SeLU
Normal	---
Zero	---



Mathematical computation of hidden layer

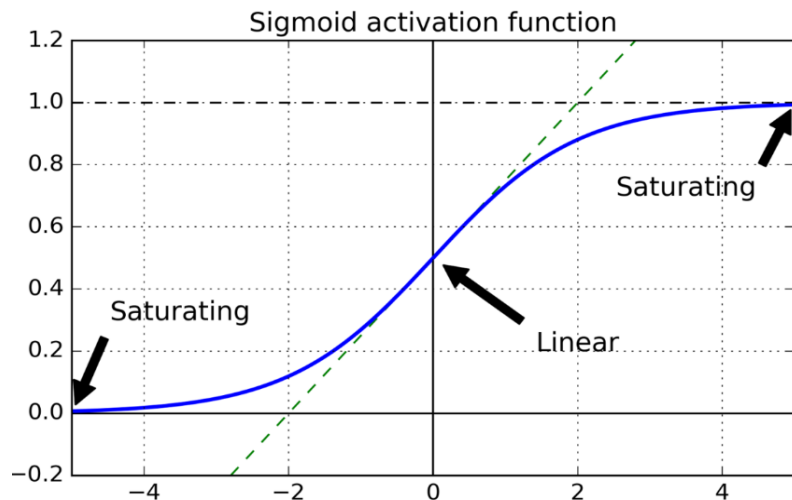
$$\mathbf{L} = \begin{bmatrix} 4 & 1 \\ 4.4 & 0.3 \\ 7.5 & 9 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 8 \\ 5.6 \\ 43.5 \end{bmatrix}$$

Forward Propagation (FP)

Vanishing and Exploding Gradient Problem



The problem where the gradient get **smaller** (vanishing) or **bigger** (exploding)



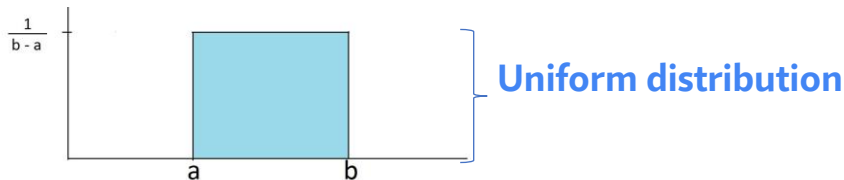
$\text{Var}(\text{input}) \gg \text{Var}(\text{output})$

The imbalanced variance is the main problem

All need to be set properly in order for your voice to come out loud and clear at the end of the chain. NNs are equal!!

Forward Propagation (FP)

Xavier Uniform Initialization

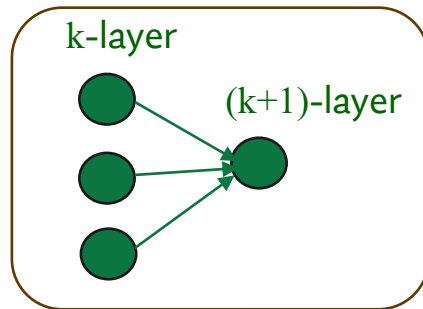


$$1 \quad \text{Var}[w_i] = \frac{2}{n^k + n^{k+1}}$$

$$2 \quad w_i \sim \mathcal{U}(-a, a) \rightarrow \text{Var} = \frac{1}{12}(a - (-a))^2$$

$$\text{Var} = \frac{a^2}{3}$$

$$3 \quad a = \sqrt{\frac{6}{n^k + n^{k+1}}}$$



The variance of weight distribution is the key point to understanding the Xavier Initialization

Algorithm 1 Xavier uniform weight initialization

Require: $k \geq 2$

$n_k \leftarrow$ number of neurons in k -layer
 $n_{k+1} \leftarrow$ number of neurons in $(k+1)$ -layer
weights $\leftarrow$ zero matrix initialization
for $i = 0$ to n_k **do**
 for $j = 0$ to n_{k+1} **do**
 weight $\leftarrow \mathcal{U}(-a, a)$
 weights[i, j] $\leftarrow$ weight
 end for
end for

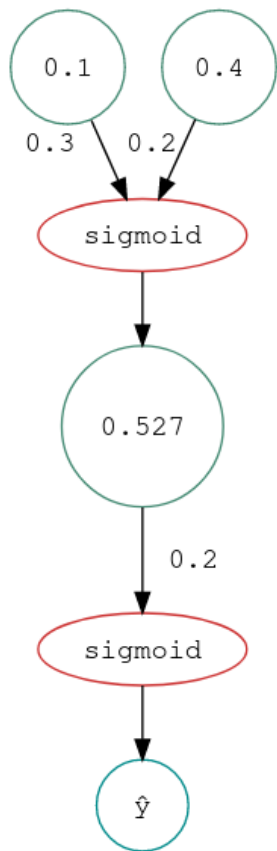
Backpropagation

Dataset

x1	x2	y
0.1	0.4	1



① Compute the output $\hat{y}$ by the forward propagation



$$h = \sum_i w_i x_i = 0.1 \times 0.3 + 0.4 \times 0.2$$

$$h = \sigma(h) = \text{sigmoid}(0.11)$$

$$h = \frac{e^{0.11}}{e^{0.11} + 1}$$
$$h = 0.527$$

$$\hat{y} = \sigma(\mathbf{w}_o h)$$

$$\hat{y} = \text{sigmoid}(0.2 \times 0.527)$$

$$\hat{y} = \frac{e^{0.105}}{e^{0.105} + 1}$$

$$\hat{y} = 0.526$$

Objective Function

$$e(\mathbf{w}, x, y) = (y - \hat{y})\sigma(\mathbf{w}x)$$

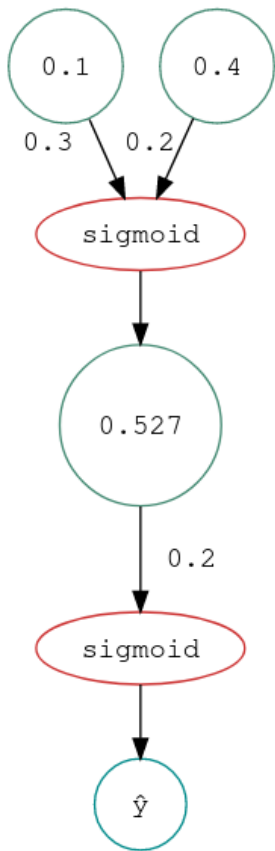
Sigmoid derivative

$$\sigma'(x) = \text{sigmoid}'(x)$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$



Backpropagation



② Start the backward pass computing the error term

$$\frac{\partial e(\mathbf{w}_o, x, y)}{\partial \mathbf{w}_o} = \delta^o = (y - \hat{y})\sigma'(\mathbf{w}_o h)$$

- For the output weight

$$\delta^o = (1 - 0.526)(0.526)(1 - 0.526)$$

$$\delta^o = 0.118$$

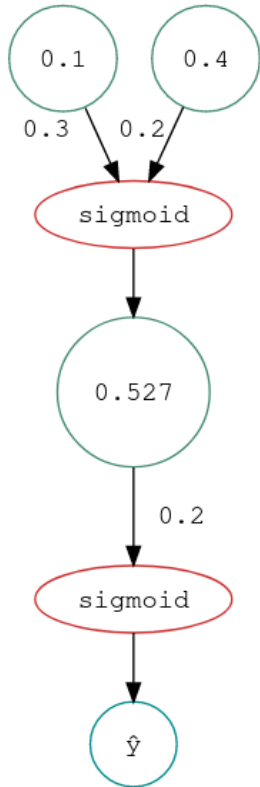
- For the hidden unit weight

$$\delta^h = \mathbf{w}_o \delta^o \sigma'(h)$$

$$\delta^h = 0.2 \times 0.118 \times (0.527)(1 - 0.527)$$

$$\delta^h = 0.0058$$

Backpropagation



③ Update all the weights (Delta rule) "See also Hebb Rule [2]"

$$\Delta w = \eta(\text{learning rate}) \times \delta(\text{error term}) \times a(\text{neuron input})$$

- **For the output weight** ($\eta = 0.5$)

$$\Delta w_o = 0.5 \times 0.118 \times 0.526$$

$$\Delta w_o = 0.031$$

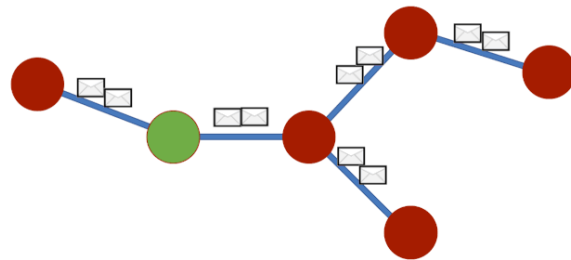
- **For the hidden unit weight** ($\eta = 0.5$)

$$\Delta w_1 = 0.5 \times 0.0058 \times 0.1$$

$$\Delta w_1 = 0.00029$$

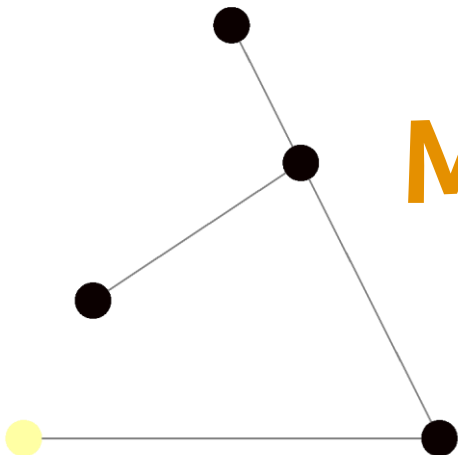
$$\Delta w_2 = 0.5 \times 0.0058 \times 0.4$$

$$\Delta w_2 = 0.0011$$



Message Passing on Graphs

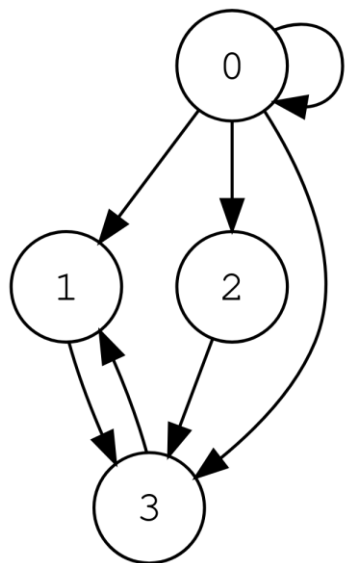
"In the simplest form, is just the multiplication between an adjacency matrix and the nodes features of a graph"



Message Passing on Graphs

The key concept is propagating the information from one node to another using the adjacency matrix of the graph

Graph Representation



Graph Adjacency Matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Node Features

$$\mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

Simple Message Passing

$$\mathbf{H} = \mathbf{A}\mathbf{X}$$

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

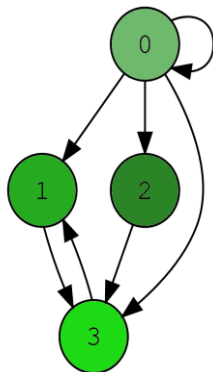
Message Passing on Graphs

Computation

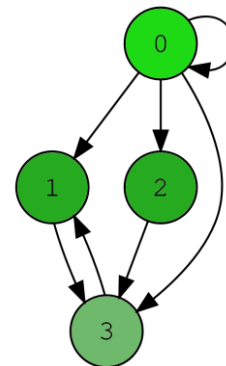
Node Features

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \times 1 + 1 \times 2 + 1 \times 3 + 1 \times 4 \\ 0 \times 1 + 0 \times 2 + 0 \times 3 + 1 \times 4 \\ 0 \times 1 + 0 \times 2 + 0 \times 3 + 1 \times 4 \\ 0 \times 1 + 1 \times 2 + 0 \times 3 + 0 \times 4 \end{bmatrix} = \begin{bmatrix} 10 \\ 4 \\ 4 \\ 2 \end{bmatrix}$$

Graph Adjacency Matrix



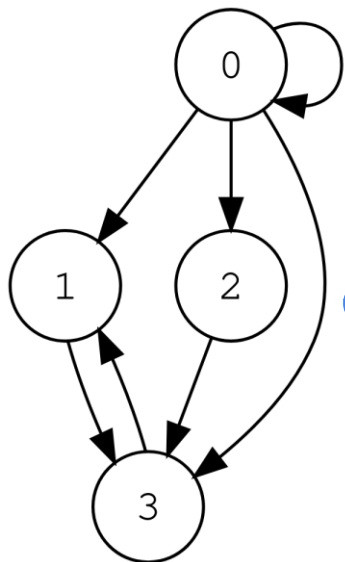
After Message Propagation



Aggregated Features

Message Passing on Graphs

Graph Representation



Graph Adjacency Matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Node Features

$$\mathbf{X} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

Inverse of Degree Matrix

$$\mathbf{D}^{-1} = \begin{bmatrix} 0.25 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The key concept is propagating the information from one node to another using the adjacency of the graph

Average Message Passing

$$\mathbf{H} = \mathbf{D}^{-1} \mathbf{A} \mathbf{X}$$

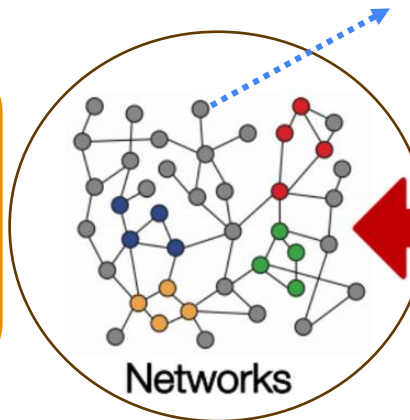
$$\mathbf{H} = \begin{bmatrix} 0.25 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

Graph Neural Networks (GNN)

"Graphs are ubiquitous in our world, network communication, social networks, chemistry molecules ... well, GNN can processed it"

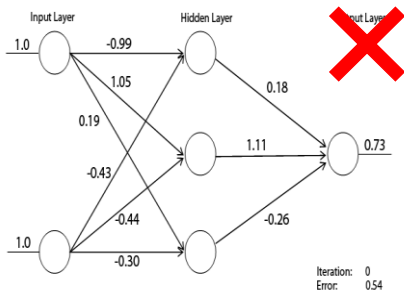
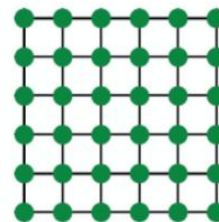
Graph Neural Networks

Are the classical ANN models capable of handling graphs (non-Euclidean) data?

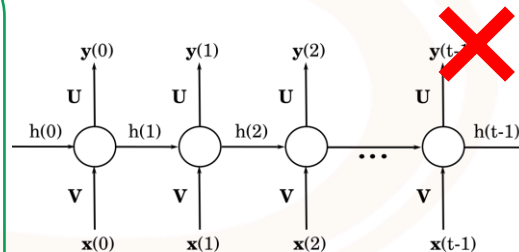


Non-euclidean data

VS.



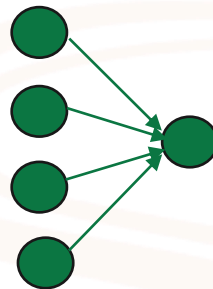
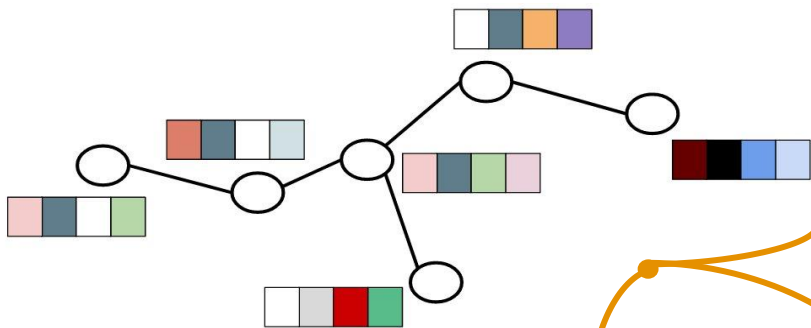
Individually, these models can't handle well the complexities inherent to the relation between the nodes and their connecting edges



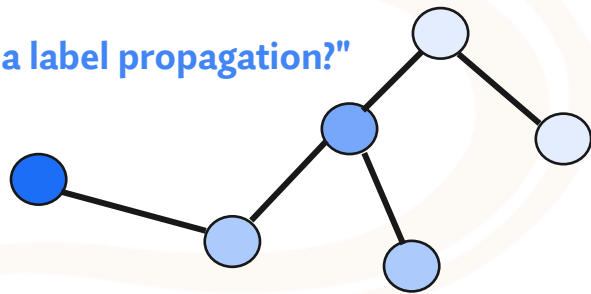
Graph Neural Networks

How to process non-Euclidean data

"Use a simple classical neural network model?"



"Perform a label propagation?"



"Use a graph neural network (GNN)?"

Graph Convolutional Networks (GCN)

**"The cornerstone graph neural network
model"**

Graph Convolutional Neural Networks

Mathematical Description

$$\mathbf{H}^{(l+1)} = f(\mathbf{H}^{(l)}, \mathbf{A})$$

Message Passing

Basic Neural Network
Layer Definition

$$f(\mathbf{H}^{(l)}, \mathbf{A}) = \sigma(\mathbf{A}\mathbf{H}^{(l)}\mathbf{W})$$

Simplest GCN Definition

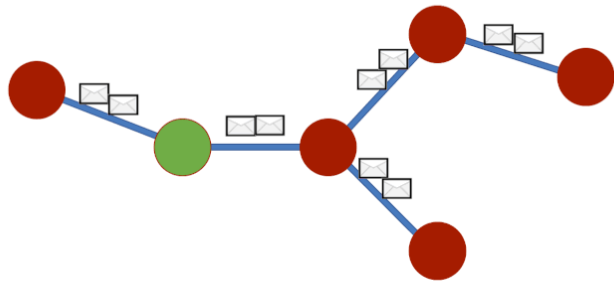
Where:

$\mathbf{H}^{(l)}$ – NN output of l -th layer

$\mathbf{A}$ – Adjacency Matrix

$\mathbf{W}$ – Weight Matrix

Graph Convolutional Neural Network (GCN)



It is a GNN model aimed to use for classification tasks. It was created with a similar intuition as Convolutional Neural Networks (CNN)

$$g_{\theta} * x = \mathbf{U} g_{\theta} \mathbf{U}^{\top} \quad \left. \vphantom{g_{\theta} * x = \mathbf{U} g_{\theta} \mathbf{U}^{\top}} \right\} \text{Complexity way to perform convolution on graphs}$$

$$\mathbf{H} = \hat{\mathbf{D}}^{-\frac{1}{2}} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-\frac{1}{2}} \mathbf{X} \mathbf{W}$$

Approximation

Normalized Adjacency Matrix

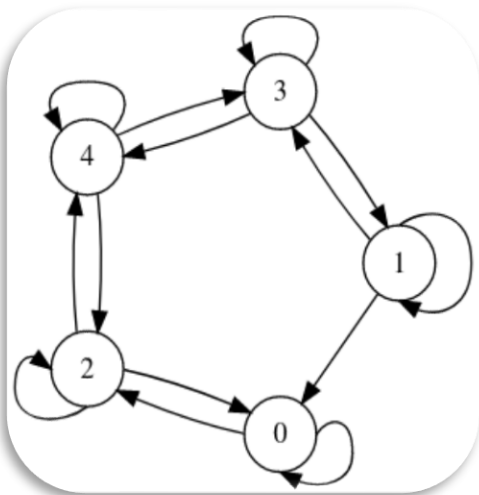
$\mathbf{A} + \mathbf{I}$

The starting point comes from a spectral graph convolution, yet introduces some simplifications

Graph Convolutional Neural Networks

Modeling GCN FP with 1 Hidden Layer for Node Classification

Considering as input the below graph with 5 nodes



Which the adjacency matrix is given by

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

Graph Convolutional Neural Networks

Modeling GCN FP with 1 Hidden Layer for Node Classification

Each node has 4 features

It is also considered a weight matrix and implicit bias given by

$$\mathbf{H}^{(0)} = \mathbf{X} = \begin{bmatrix} 2 & 1 & 0 & 0 \\ 0 & 1 & 0 & 3 \\ 1 & 0 & -1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{W} = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & -1 & 1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 0 & 0 & -1 \end{bmatrix}$$

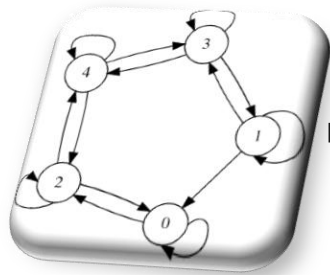
Weight Matrix

Bias

Graph Convolutional Neural Networks

Modeling GCN FP with 1 Hidden Layer for Node Classification

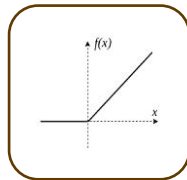
Input Graph



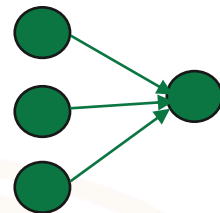
Forward Propagation



Non-linear Activation Function



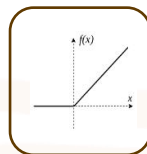
Multilayer Perceptron



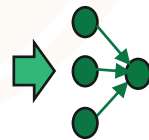
"For each node in the graph: classify"

$$\mathbf{A}\mathbf{H}^{(0)}\mathbf{W}$$

ReLU



$$\begin{bmatrix} 4 & 1 & 2 \\ 4 & 0 & 5 \\ 5 & 1 & 3 \\ 2 & 0 & 4 \\ 2 & 0 & 2 \end{bmatrix}$$



Message Passing Based on Neural Networks

"The use of neural network models to perform the well-known message passing"

Message Passing Using an RNN

RNN Forward Propagation Law

$$\mathbf{h}_t = \sigma(\mathbf{h}_{t-1}\mathbf{W} + \mathbf{x}_t\mathbf{V})$$

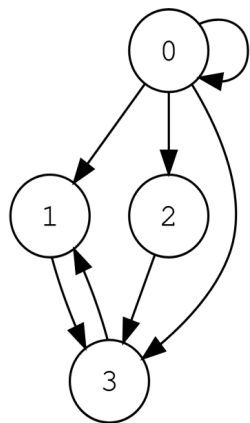
$$\mathbf{y}_t = \mathbf{h}_t\mathbf{U}$$

Weights and hidden states

$$\mathbf{W} = \begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \quad \mathbf{V} = \begin{bmatrix} 6 \\ 4 \end{bmatrix}$$

$$\mathbf{U} = \begin{bmatrix} -1 & 1 \end{bmatrix}$$

$$\mathbf{h}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



Node Features

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

In $t=1$ we have, considering a ReLU activation function:

$$\mathbf{h}_1 = \text{ReLU} \left(\begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 1 \times \begin{bmatrix} 6 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 6 \\ 4 \end{bmatrix}$$

$$\mathbf{y}_1 = \begin{bmatrix} 6 \\ 4 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = -2$$

Message Passing Using RNNs

All Computations

At t=1:

$$\mathbf{h}_1 = \text{ReLU} \left(\begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} + 1 \times \begin{bmatrix} 6 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 6 \\ 4 \end{bmatrix}$$

$$\mathbf{y}_1 = \begin{bmatrix} 6 \\ 4 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = -2$$

At t=2:

$$\mathbf{h}_2 = \text{ReLU} \left(\begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \begin{bmatrix} 6 \\ 4 \end{bmatrix} + 2 \times \begin{bmatrix} 6 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 34 \\ 64 \end{bmatrix}$$

$$\mathbf{y}_2 = \begin{bmatrix} 34 \\ 64 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = 30$$

At t=3:

$$\mathbf{h}_3 = \text{ReLU} \left(\begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \begin{bmatrix} 34 \\ 64 \end{bmatrix} + 3 \times \begin{bmatrix} 6 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 308 \\ 412 \end{bmatrix}$$

$$\mathbf{y}_3 = \begin{bmatrix} 308 \\ 412 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = 104$$

At t=4:

$$\mathbf{h}_4 = \text{ReLU} \left(\begin{bmatrix} 1 & 4 \\ 8 & 2 \end{bmatrix} \begin{bmatrix} 308 \\ 412 \end{bmatrix} + 4 \times \begin{bmatrix} 6 \\ 4 \end{bmatrix} \right) = \begin{bmatrix} 1980 \\ 3304 \end{bmatrix}$$

$$\mathbf{y}_4 = \begin{bmatrix} 1980 \\ 3304 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = 1324$$

Hidden States

6	34	308	1980
4	64	412	3304

Outputs

y	-2	30	104	1324
---	----	----	-----	------

Programming a GNN Model

"A Hands-on approach in how to use the main packages to create a GNN model"

Programming a GNN Model

Frameworks



Deep Graph Library



Pytorch Geometric



TensorFlow GNN 2.0



iGNNition

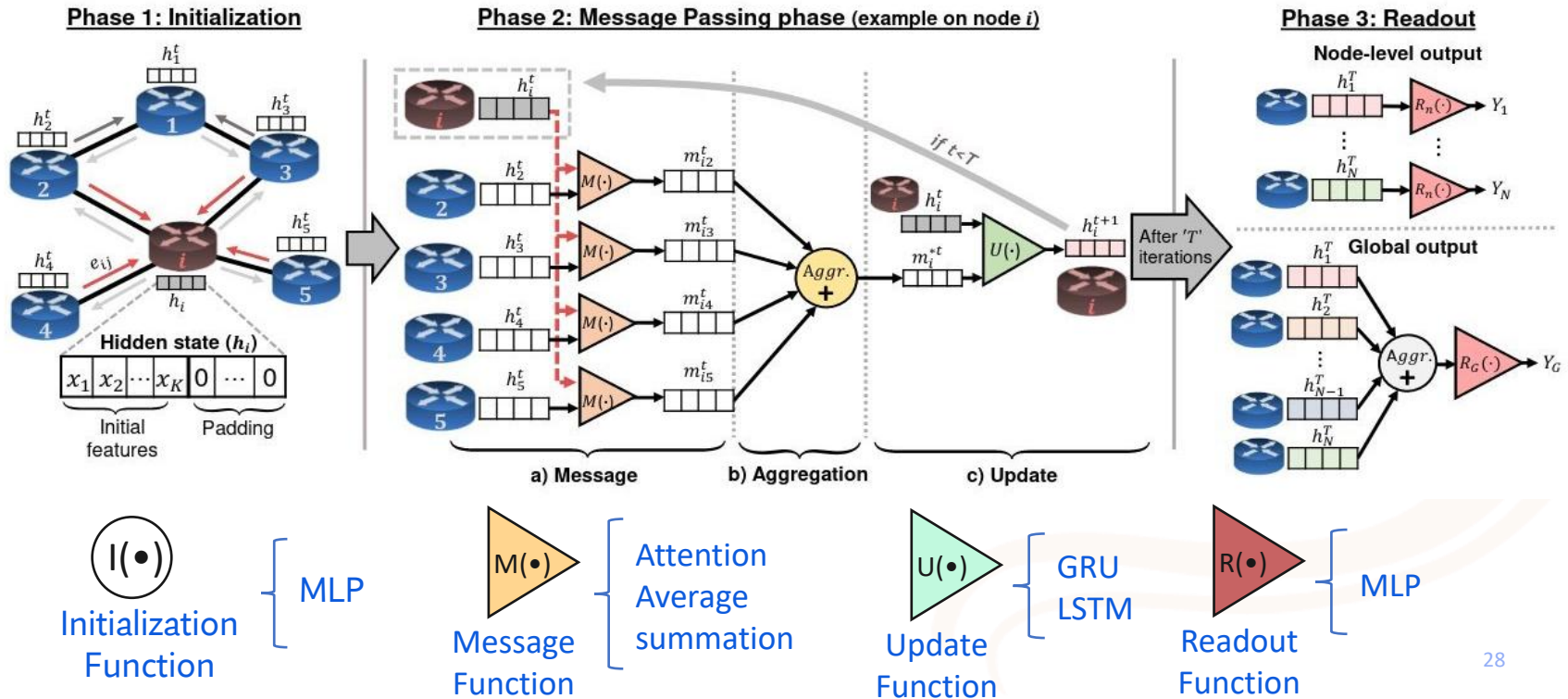
iGNNition

An Application for Network Communication Modeling

An Application for Network Communication Modeling

Message Passing Neural Networks (MPNN)

$$\text{MPNN} \supseteq \text{GCN}$$

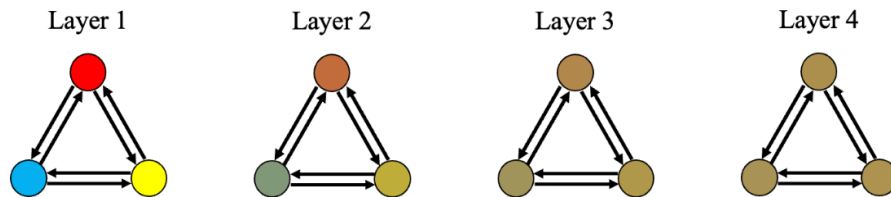


An Application for Network Communication Modeling

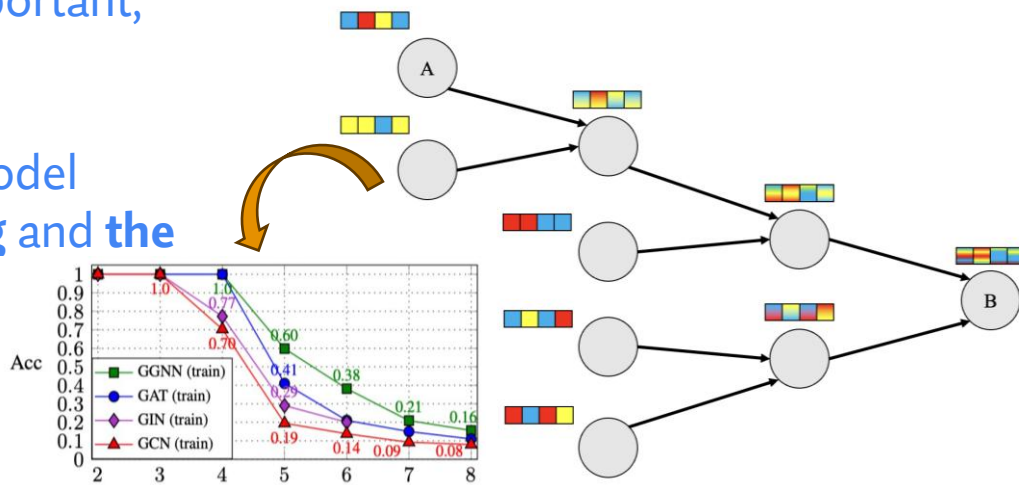
Hyperparameters

- The size of hidden state vectors;
- The numbers of iterations.
- Setting the correct numbers for these hyperparameters is extremely important, otherwise, convergence to a good performance is unlikely;
- The main issue associated with model divergence is the **over-smoothing** and the **over-squashing**:

Over-smoothing Problem



Over-squashing Problem



An Application for Network Communication Modeling

Modeling Network with ML Models

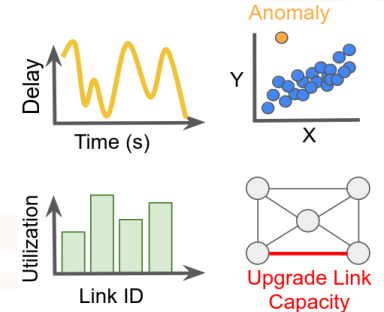
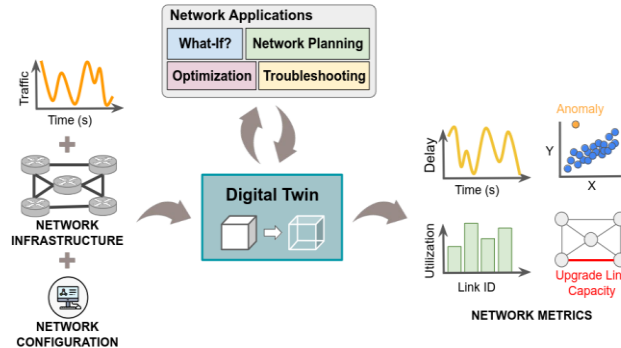
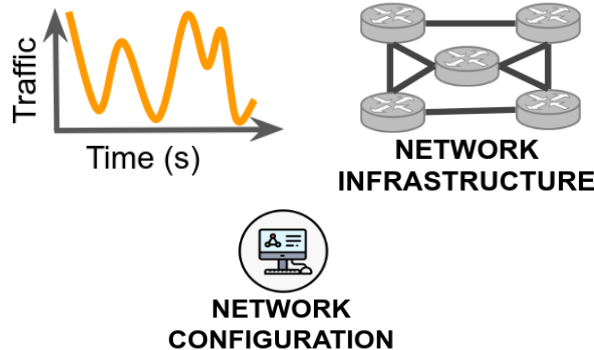
Use the network communication inputs to mimic complex environments intended to retrieve interest metrics



Use this interest metrics to evaluate the communication network under certain conditions (*what-if analysis*)



Use the evaluation results to recommend modification in the real environments



An Application for Network Communication Modeling

Pipeline

